

SD TCL - Files and Records

Making, configuring and removing files; copying, renaming, deleting and dumping the records inside them.

This page covers the verbs that act on a whole file or on the records in it, from the command line. What a program does with the same files is SD Basic - File Handling; this is the part you type.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case, which is what this port uses on disk. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The two kinds of file, and what they look like in Explorer

Both kinds are a directory on Windows. This surprises people who expect a database file to be one file.

dynamic a directory holding components named `%0`, `%1`, ... The records are hashed across them and none of it is readable from outside SD

directory a directory holding **one ordinary file per record**, named by the record id

`voc`, `$ipc` and `dict.dic` are dynamic. `messages`, `accounts` and `bp` are directory files — which is why you can open a BASIC program in Notepad and why `gp1.bp` is readable in the source tree.

A dictionary is a separate file that travels with the data file. Verbs that can act on either take `dict` in front of the file name; with no `dict` they mean the data portion.

The shipped file names are lower case on disk in this port. `voc`, `bp`, `newvoc`, `accounts`, `messages` and the rest were renamed. NTFS matches without being asked, so this changes nothing about what you can type — but it is what you will see in Explorer and in the path text SD reports back.

Making and removing files

```
create.file {dict|data} voc.name directory
create.file {dict|data} voc.name dynamic {parameters}
create.file voc.name using dict filename
```

`create.file` makes the file **and** the VOC record that names it. Without `dict` or `data` it makes both portions. `using dict` *filename* takes the dictionary from an existing file instead of creating an empty one, and `no.query` suppresses the confirmation.

Dynamic files take parameters, all optional:

<code>minimum.modulus</code> <i>n</i>	the floor the file will not shrink below
<code>group.size</code> <i>n</i>	group size in 1 KB units
<code>large.record</code> <i>n</i>	the size above which a record is stored separately
<code>split.load</code> <i>n</i> · <code>merge.load</code> <i>n</i>	the load factors that trigger growth and shrink
<code>version</code> <i>n</i>	file format version
<code>directory</code> <i>path</i>	put the file at <i>path</i> instead of in the account
<code>no.case</code>	record ids are case-insensitive
<code>no.resize</code>	disable automatic resizing

```
delete.file {dict|data} voc.name {force} {no.query}
clear.file {dict|data} file
```

`delete.file` removes the file and its VOC record. `force` **is needed if the file is not empty**. `clear.file` empties a file and keeps it, along with its dictionary and its indexes.

Changing a file after it exists

```
configure.file {dict} voc.name {parameters}
configure.file {dict} voc.name default
```

The parameters are the `create.file` ones, plus `immediate` to do the work now rather than lazily, and `dynamic` or `directory` to change the file's kind. `default` puts the settings back.

Two of them restructure the whole file. Changing `group.size` or `version` requires a full restructure; the rest take effect without one. Plan for the time and the disk on a large file.

`no.case` / `case` and `resize` / `no.resize` are the pairs that turn a setting on and off. For a directory file, `binary` turns on mark mapping.

Looking at a file

```
analyse.file {dict} file {statistics} {lptr}
```

Without `statistics` it reports the shape: the account, the file name, the path, the type, the group size, the large-record size, the minimum and current modulus, the load factors and the size in bytes. With `statistics` it walks the whole file and adds record counts, per-group minimum, maximum and average, and a size distribution. `statistics` **reads every group**, so it is not free on a big file.

`analyze.file` is the same verb under the other spelling.

<code>fstat filename on</code>	start collecting, clearing the counters
<code>fstat filename {lptr}</code>	show what has been collected
<code>fstat filename off</code>	stop collecting
<code>fstat global {lptr}</code>	the global counters
<code>fstat reset</code>	clear the global counters
<code>fstat</code>	the periodic global summary

`fstat` **only works on dynamic files**. A directory file named on the command line gets a warning; one arriving through a select list is skipped silently. Where a file name is taken, a select list or several names may be given instead.

```
hsm {on|off|display} {user n}
```

The Hot Spot Monitor. `display` is the default.

```
list.files
```

Reports how many files are open now, the peak for the session and the configured limit, then lists them:

```
Number of files open = 2.  Peak = 6.  Limit = 80.
```

The limit is the `NUMFILES` configuration parameter. **The peak is what matters when you are sizing it** — the current figure moves constantly and tells you almost nothing.

Reaching a file in another account

```
set.file account file.name pointer.name
```

This writes a **Q-pointer** into your VOC — an indirect pointer naming another account and a file in it. After it, *pointer.name* is usable wherever a file name is, and the data stays where it is.

The account must be in the accounts register, and the name is folded to upper case before the register is read. If it is not there you get *Account name '...' is not in register* and nothing is written.

Read what `set.file` prints, not just whether it printed. Its refusals echo the names you passed in, so a script that checks its output for the pointer name it asked for will find that name in the refusal as well as in the success. Check for the success wording, and treat *not in register* and *not found* as disqualifiers.

Copying, deleting and renaming records

```
copy from {dict} src.file {to {dict} tgt.file} {s1{,t1} {s2{,t2}}...}  
copy from {dict} src.file {to {dict} tgt.file} all
```

With no `to` clause the copy is **within the source file**, which is how you duplicate a record under a new id. Each `s`, `t` pair is a source id and the id to give the copy; a bare `s` keeps the id. Options:

<code>overwriting</code>	replace records that already exist in the target
<code>updating</code>	copy only records that already exist in the target
<code>deleting</code>	delete the source record after a successful copy
<code>reporting</code>	list what was copied
<code>binary</code>	force binary mode when one side is hashed and the other a directory

`@system.return.code` comes back as the **number of records copied**, or negative on error.

```
copyp {dict} filename id... {(options)}
```

The Pick-style copy. Its options are single letters in a trailing bracket: `d` delete source, `i` suppress the id, `n` no pagination, `o` overwrite, `p` to the printer, `s` suppress field numbers, `t` to the terminal.

```
delete {dict} file.name {record.name} {no.query}
delete {dict} file.name all
```

`@system.return.code` is the **number of records deleted**, or negative on error.

```
cname old.file.name to new.file.name
cname {dict} file.name old.record.name to new.record.name
```

A comma may be used instead of `to`. `rename` is the same verb under the other name. `@system.return.code` is `0` on success and `-1` if the arguments were wrong.

`cname` **on a file renames the VOC record too**, which is the point — the name you type and the name on disk stay together.

Seeing what is actually in a record

```
ct {dict} file.name {record.name... | *} {options}
dump {dict} file.name {record.name... | *} {options}
```

Both are the same program. `ct` is a field-by-field listing; `dump` reports the same data in binary. Options: `hex` for hexadecimal, `binary` for a full-width report that is not field-based, `no.query` to skip the prompt when a select list is in use, and `lptr {n}` to send it to a print unit.

`ct` **is the one verb here a standard account has**; `dump` is withheld. They run the same code, and the line between the tiers runs between the two VOC records rather than through the program.

Triggers

```
set.trigger file.name trigger.name {modes}
set.trigger file.name ""
set.trigger file.name
```

Set, remove, or display a file's trigger function. An empty string in quotes removes it; no argument at all displays it.

Who has these verbs

Almost everything on this page is withheld from a standard account. That is deliberate: these are the verbs that change data in bulk.

standard

`ct` `list.files` `set.file`

programmer

`analyse.file` `analyze.file` `clear.file` `cname` `configure.file` `copy` `copyp`
`create.file` `delete` `delete.file` `dump` `fstat` `hsm` `rename` `set.trigger`

A standard account can therefore look at a record and point at a file, and cannot create, empty, delete, copy or rename anything. An account that does not have a verb does not have the VOC record for it, so the name is not recognised at all rather than refused.

What is not here

Indexes are their own subject and are not on this page. For what an index is and what it does to a file, see SD Basic - Alternate Key Indexes.

Selecting records is the query processor's job, not these verbs'. `copy`, `delete` and `ct` all accept a select list built by `select` or `sselect`, and that is how they are usually driven over more than a handful of ids.

`reformat` and `sreformat` rewrite records through a dictionary and are run by the query processor, so they are documented with it.

There is no verb that repairs a damaged file. `analyse.file statistics` will tell you a file is badly overflowed; `configure.file` is what acts on that.

See also

SD TCL - The Command Processor · SD Basic - File Handling · SD Basic - Select Lists.